

HPL HPC Assignment Pipeline

Technical architecture — pipeline stages, the Knowledge Bank, and how the pieces are deployed

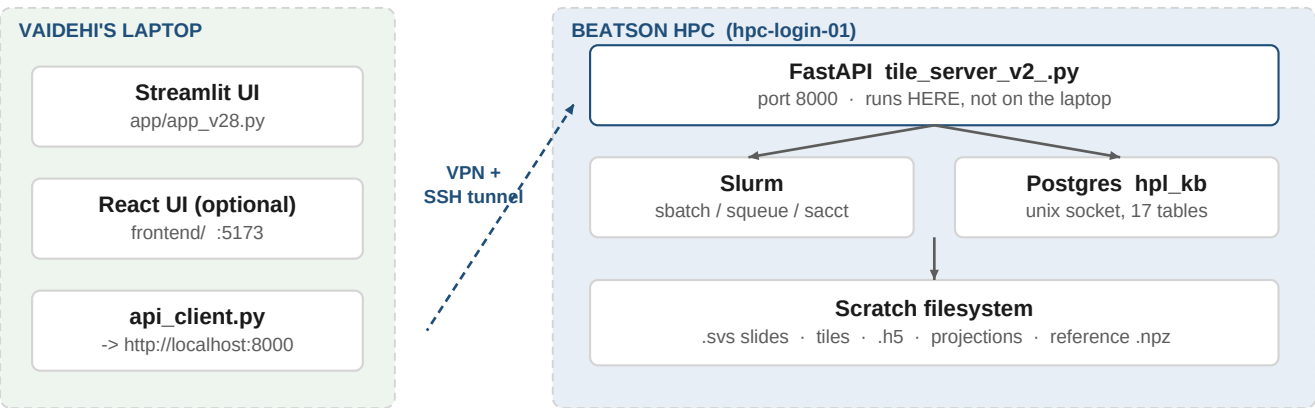
Version	2026-08-26
Scope	backend/ (FastAPI + Slurm submitters), app/ (Streamlit UI), frontend/ (React UI), Postgres hpl_kb
Cluster	Beatson HPC, University of Glasgow — Slurm, Singularity, NVIDIA H200/H100/A100
Current UI	app/app_v28.py · current server: backend/tile_server_v2_.py
Reference	hpc_reference_leiden_2p5_fold2.npz — LATTICeA, leiden 2.5, fold 2, 2.5M tiles, 71 clusters
Source	docs/make_architecture_pdf.py — regenerate rather than edit

What this system does

Whole-slide H&E images are cut into tiles, each tile is turned into a 128-dimensional vector by a frozen self-supervised encoder, and each vector is assigned to one of 71 **histomorphological phenotype clusters** (HPCs) by exact k-nearest-neighbour search against a reference built from the LATTICeA cohort. The results land in a Postgres **Knowledge Bank** that a slide viewer, an HPC explorer and a natural-language chatbot read.

The pipeline is six stages. The first four submit Slurm jobs and produce files; the last two write to the Knowledge Bank from inside the API process. Every stage is gated in the UI on the previous stage having produced a *valid* output — not merely on it having run.

Deployment — what runs where



The API server runs on the HPC because it shells out to sbatch and opens .svs files from scratch. The laptop runs only the UI.

The API server runs **on the HPC login node**, not on the laptop. It shells out to sbatch, squeue and sacct, opens .svs files directly from scratch, and reaches Postgres over a local socket. The laptop runs the UI only, and reaches the API through the university VPN plus an SSH tunnel to port 8000. Deploying a change to backend/ therefore means copying it to the cluster and restarting the server — running it locally is not enough.

Two access paths exist, and the split matters when planning a change. Most of the UI goes through `api_client.TileServerClient` over HTTP. The chatbot and three viewer helpers (`load_hpc_titles`, `load_survival_coefficients`, `slides_for_hpc_from_kb`) query Postgres *directly* via SQLAlchemy, bypassing the API. Those need a second tunnel to 5432. The React frontend has no direct database path at all — it is HTTP-only, which is why the chatbot was deliberately left out of that port.

The pipeline, stage by stage



Stages 1-4 submit Slurm jobs. Stages 5 and 6 run inside the API process and commit in one transaction each.

Stage	Runs where	Reads	Writes	State tracked
1 Tiling submit_mask_tile_slurm.py	Slurm array, 1 CPU/task	raw .svs	tiles on disk + per-slide _tile_metadata.csv	job_id, manifest; completion recomputed
2 Packaging make_hpl_hdf5.py	Slurm, 8 CPUs	tiles + CSVs	one gzip .h5 per dataset	h5_job_id, h5_output_path
3 Feature extraction submit_feature_extraction.py	Slurm GPU, Singularity	packaged .h5	projections .h5 (z_latent, h_latent)	extraction_job_id, checkpoint
4 Classification submit_cluster_assignment.py	Slurm CPU, faiss	projections + reference .npz	assignments .csv	assignment_job_id, assignment_vote
5 Registration register_dataset.py	in the API process	raw slides, Stage 1 CSVs, packaged .h5	wsi_registry, wsi_metadata, dataset_config, tile_coordinates, tile_registry	registration_done
6 KB load load_hpc_assignments.py	in the API process	assignments .csv	tile_registry.hpc_id, hpl_profile_*	kb_load_done

Stage 1 does not write tile_coordinates. It writes a per-slide CSV to disk. Nothing puts those rows into Postgres until Stage 5. The project documentation asserted otherwise until 2026-08-26, and that mistake is what kept the registration gap invisible: everyone believed the coordinates table filled itself.

Why Stage 5 exists

load_hpc_assignments.load() only ever UPDATES tile_registry.hpc_id — it never creates a row. For a cohort that has never touched the Knowledge Bank there is nothing to update, so Stage 6's match rate is 0% by construction and it refuses. The refusal is correct and its message names a match rate, which reads like a slide-naming bug. It is in fact a missing step.

Stage 5 creates the identity rows Stage 6 needs. It is gated on Stage 2, not Stage 4: it reads tile identity out of the packaged .h5 and the raw slides and needs no cluster labels, so it can finish while the GPU work is still queued.

All five tables are written in **one transaction**. A registration that wrote `tile_registry` but not `ws_i_registry` would pass every check the pipeline has — Stage 6 loads, the aggregates refresh, every table grows — and the cohort would still be invisible in the viewer, because `_open_slide()` resolves slide paths from `ws_i_registry` alone.

The Knowledge Bank

Postgres `hpl_kb`: 17 tables, one view, 45 indexes. Column-level definitions live in `backend/kb_live_schema_2026-08-26.txt`, transcribed from `\d` against the live database.

FILLED BY A RUN	STATIC REFERENCE	NOTHING FILLS THESE
tile_coordinates Stage 5	hpc_dictionary 71 clusters	tile_hpc_heatmap READ LIVE - 149 MB
tile_registry Stage 5 + 6	hpc_malignant_details 27 rows	h_latent_vectors no reader - 4.4 GB
ws_i_registry Stage 5	hpc_non_malignant_details 44 rows	tile_hpc_heatmap_old legacy - 141 MB
ws_i_metadata Stage 5 (opt-in)	hpc_survival_analysis Cox coefs	slide_metadata (view) no reader
dataset_config Stage 5		
hpl_profile_summary Stage 6		
hpl_profile_proportion Stage 6		
slide_hpc_membership Stage 6		
slurm_dataset_runs the server		
slurm_dataset_run_jobs the server		

`tile_hpc_heatmap` is the one open gap with a live reader: the server loads it at startup and nothing has ever written it.

The join key

`slide_tile` is "`<slides>_<tiles>`" upper-cased — `TCGA-55-7574-01Z-00-DX1_18_15.JPEG`. It is the primary key of both `tile_coordinates` and `tile_registry`, and the two halves of the pipeline disagree about the tile name: Stage 1 writes `24_10.jpeg`, packaging writes `24_10`. Both sides go through `backend/slide_naming.py` so a key built from one can match a row registered from the other.

A grep will not find every reader

`app/hpc_chat_handlers_v23.py:334` does not query tables by name. It enumerates the whole database with `insp.get_table_names()`, keeps every table carrying an `hpc_id` or `dominant_hpc` column — skipping only `hpc_dictionary` and `h_latent_vectors` — and renders up to five matching rows straight to the user. So such a table is answered out of the chatbot without ever appearing in a query, and a `grep` for its name finds nothing. That is how `slide_hpc_membership` looked unreferenced while serving 19,493 stale rows from an older cohort. **Before concluding a table is dead, check whether it has an `hpc_id` column.**

Aggregates are what the UI actually reads

`hpl_profile_proportion` and `hpl_profile_summary` are derived from the same assignment CSV as `tile_registry`, and they — not `tile_registry` — are what the chatbot and the HPC panels query. Writing tiles without refreshing the aggregates leaves the UI internally inconsistent with nothing to signal it.

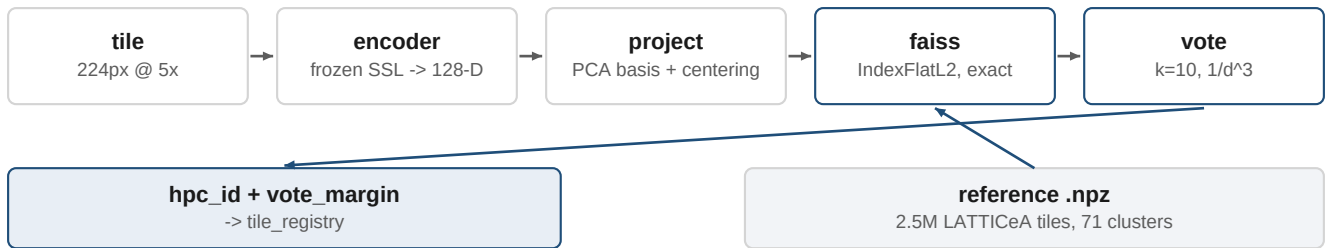
Two schema facts worth knowing before extending this. `hpl_profile_summary`'s uniqueness constraint is on (samples, slides) and does **not** include `dataset_id`, so two cohorts holding the same slide id cannot both have a summary row. And `slide_hpc_membership` has no `dataset_id` column at all, so it cannot be scoped per cohort even in principle.

Schema provenance

Only nine of the seventeen tables had a `CREATE TABLE` anywhere in git until 2026-08-26, and `dataset_id` — the column every cohort guard is scoped by — appeared in no migration at all. `backend/migrate_kb_base_tables.sql` now covers the missing eight. `schema.sql` in the repository root is a stale 2025-10-23 dump that disagrees with the live `tile_registry` on its primary key and on `hpc_id`'s type; it carries a header saying so.

```
psql -h <socket> -d hpl_kb -f backend/migrate_all.sql
# idempotent; runs every migration in dependency order
```

The classifier



Re-voted at `k=25` when `vote_margin < 0.15`. Closed-book accuracy on unseen slides: 97.33%.

Search is **faiss-only and exact** — `IndexFlatL2`, no numpy fallback and no approximate index. An approximate index was measured against the real reference: it agreed on the nearest neighbour 33% of the time for no speed gain at this reference size, which would have put an approximation inside the one number the pipeline is judged on.

Measurement	What it actually asks	Result
Leave-one-out	Does k-NN recover a LATTICeA tile's own label, given the rest of LATTICeA (slide-mates included)?	97.23% at the settled vote
Closed-book slide holdout	A LATTICeA slide the reference has never seen — 20 slides, 16,468 tiles held out	97.33% ± 0.60 between-slide SD
TCGA acceptance test	Do we reproduce <code>sc.tl.ingest</code> 's cross-cohort transfer of LATTICeA clusters onto TCGA? 499,108 tiles	99.619% agreement
A new cohort (e.g. Radiogenomics)	No ground truth exists. Probe by distance instead — <code>backend/cohort_shift.py</code>	unmeasured by construction

The settled vote: `--k 10 --distance-weighted --distance-power 3`, re-voting at `k=25` for tiles whose `vote_margin` falls below 0.15. It searches once at the wider `k` and votes on a prefix, so the base vote and the re-vote see the same neighbours by construction.

Leave-one-out accuracy and the acceptance test move in *opposite* directions on purpose. The acceptance test asks whether we reproduce `sc.tl.ingest`, which is an unweighted majority vote — so distance weighting makes the classifier deliberately less like `ingest`. Run the acceptance test twice: at the default, to confirm the ≥99% gate that protects against projection and centering bugs; and at the tuned preset, to see the size of the intended divergence. A drop at the *default* is a real defect.

Confidence is calibrated, and Stage 6 can use it

`vote_margin` stratifies accuracy cleanly even off-reference: on held-out slides, tiles above margin 0.75 were 100% correct (70% of all tiles) while those below 0.10 were 59.4%. Stage 6's `min_margin` excludes low-confidence tiles from the per-slide aggregates only — `tile_registry` keeps every tile's own `hpc_id` and margin regardless.

The failure mode this codebase is written against

Almost nothing here fails by crashing. A wrong reference, a naming mismatch, a shard carrying its own mean, a half-merged output, a stale aggregate — each produces a file or a table of the right shape and dtype with no missing values. It passes every completeness check while every cluster id is attached to the wrong tile.

Everything below follows from that, and is worth preserving in anything added to this pipeline.

Guard	Where	What it refuses
95% match rate	<code>load_hpc_assignments.py</code>	A load where the CSV and the registry disagree about slide naming for all but a handful of tiles. The number it guards against is not 0% — that is obvious — but the 3% that reads as “it worked” in a summary line.
Unknown cluster ids	Stage 6, CLI and endpoint	Cluster ids with no <code>hpc_dictionary</code> row. Those tiles would render with no pattern or malignancy annotation.
Cohort collision	<code>register_dataset.py</code>	A tile or slide whose key already belongs to a different <code>dataset_id</code> . Refused outright, never reassigned — two cohorts claiming one tile means one of them is misidentified.
Ambiguous slide file	Stage 5	A slide id matching more than one file. Registering either would mean being silently wrong about which physical slide a cohort’s tiles came from.
Atomic publish	<code>make_hpl_hdf5.py</code>	A half-written <code>.h5</code> occupying the final path. Output is staged to <code>.partial</code> and renamed only when whole.
Flush before checkpoint	packaging	Recording tiles as done before their pixels are durable — which left permanently zero-filled holes that resume would skip past forever.
--centering query sharding	<code>assign_hpc_clusters.py</code>	Sharding without a shared precomputed mean. The mean is over <i>all</i> queries, so chunking silently changes every label.
Preview before commit	Stages 5 and 6	Any automatic write to the shared Knowledge Bank. A dry run has to be pulled up first, and the endpoint calls the same functions the CLI does rather than reimplementing the guards more loosely.

Two rules for anyone extending this. Prefer a loud refusal at submit time over a plausible result later. And when adding a test, make it prove the guard can *fail* — several bugs here were found by checking that a validator could come out bad, not that it came out good.

Operations

Running it

```
# on the HPC (inside the VPN)
cd backend && python tile_server_v2_.py          # FastAPI on :8000

# on the laptop, with a tunnel to the login node
ssh -N -L 8000:localhost:8000 -L 5432:localhost:5432 beatson-hpc
streamlit run app/app_v28.py                    # talks to localhost:8000
```

Deploying a change

backend/ must be copied to the cluster and the server restarted; app/ runs on the laptop. They are separate processes and neither picks up the other's changes. Two additional couplings are easy to miss: the package list for the Singularity container lives in `submit_feature_extraction.py`, so that file and `--bootstrap-extras` must always move together; and the encoder patch is mirrored in `backend/patches/hpl_encode-io.patch` because the HPC has its own clone of the HPL repository that the git subtree does not update.

Schema migration

```
psql -h /nfs/home/users/vpandya/databases/postgres_hpl_kb/socket -d hpl_kb \
-f backend/migrate_all.sql
# idempotent — a no-op against an up-to-date database
```

Tests

```
python3 -m pytest backend/tests -q
python3 backend/tests/test_register_dataset.py # standalone, no pytest
```

Every suite also runs standalone, which is how they run on the cluster where pytest may not be installed.

Known open items

Item	State
tile_hpc_heatmap has no writer	The server reads all 149 MB of it at startup and merges its 71 p_hpc_* columns into the tile overlay. The k-NN classifier does not produce a 71-class distribution — it produces a top-1 label and a margin. Filling this is a modelling decision, not plumbing.
Four cluster reference tables have no trustworthy DDL in git	\d hpc_dictionary, \d hpc_malignant_details, \d hpc_non_malignant_details, \d hpc_survival_analysis and \d+ slide_metadata have never been captured. Until they are, a fresh checkout still cannot build a complete schema.
The viewer assumes one scan geometry for every cohort	TILE_SIZE_NATIVE is computed from a hard-coded 0.252 mpp. dataset_config and wsi_metadata now carry the real numbers per cohort; using them is a separate change with real risk to the viewer.
No cohort has been registered against the live database	The path was verified end to end against a local PostgreSQL 16.2 — empty database, migrate, register, load — which found two bugs SQLite could not: a fresh schema that still would not build, and dataset_id never being set on the aggregates although it is NOT NULL there. Both fixed. The live database has real slide names and real scale, so preview the first cohort before committing it.